VaNi

Smart Forms System

PRD + Architecture + Plan of Action

Product: **ContractNest**
Company: **Vikuna Technologies**
Author: **Charan Kamal Bommakanti**
Date: **March 2026**
Version: **1.0**
Classification: **Internal — Confidential**

1. Executive Summary

ContractNest's Smart Forms system transforms service contract execution from paper-based checklists into an AI-powered, equipment-aware, context-sensitive digital evidence layer. Powered by VaNi (Vikuna's AI framework) and backed by a comprehensive Knowledge Tree, Smart Forms auto-generate the right checklist for the right equipment variant, with the right thresholds, adjusted for geography, climate, and industry context.

This document captures the complete product requirements, technical architecture, and phased implementation plan for building the Smart Forms capability end-to-end.

1.1 The Product Moat

No CMMS or field service tool ships with an AI that understands service contracts at a domain level. Competitors give blank form templates and say "build your own." ContractNest ships with a Knowledge Tree that knows equipment variants, spare parts, maintenance checkpoints, valid values, service cycles, and failure patterns — customized to each tenant's geography, climate, and industry. This is the moat.

1.2 Key Design Principles

VaNi recommends, user decides. At every step, the AI generates recommendations (variants, parts, checkpoints, cycles). The admin or tenant reviews, adds, removes, and confirms. The AI doesn't automate away human judgment — it accelerates it.

One skill, four modes. The Knowledge Tree Skill operates in Research mode (admin seeding), Registration mode (tenant onboarding), Form mode (composing smart forms), and Alert mode (runtime intelligence). Same data, different context.

Master + Fork. Forms default to master templates. Tenants can use as-is (70–80% of cases) or fork into their own copy for customization. Master updates flow through to non-forked users.

Context-aware everything. Geography, climate, industry, and industries served shape the research output, cycle frequencies, and alert thresholds. A tenant in humid Hyderabad serving healthcare gets different defaults than one in arid Rajasthan serving manufacturing.

2. Capabilities

2.1 Capability 1+2 (Merged): Knowledge Tree Builder

Originally conceived as two separate capabilities (master data seeding and LLM research for unknown equipment), these merge into a single unified flow. Whether the equipment exists in the catalog or not, the user experience is identical: type equipment name → disambiguation → context gathering → variants → spare parts → checkpoints → cycles → review → save.

Flow: 7 Steps

Step	Name	Description
0	**Input & Context**	User types equipment name. VaNi disambiguates ("apple = apple"). Gathers context: geography, climate, industry, industries served. Animated status messages show context-aware research.

1	**Variants**	Shows identified variants with select/deselect. User can add custom variants. For HVAC: Split AC, Cassette, Ductable, VRF, Chiller (water/air), AHU, FCU, Package, Window.
2	**Spare Parts**	Grouped by component type (electrical, mechanical, refrigerant, filters, water-side, controls, consumables). Per-variant toggle matrix — VaNi recommends, user decides. Add custom parts.
3	**Checkpoints & Values**	Two types: CONDITION (qualitative — dropdown values with severity: ok/attention/critical) and READING (quantitative — numbers with unit, normal range, amber/red thresholds). Add custom.
4	**Service Cycles**	Frequency per checkpoint, varies-by factors (climate, season, industry, equipment age), alert-if-overdue thresholds. Context-adjusted defaults.
5	**Form Preview**	Auto-composed smart form from knowledge tree. Web + mobile view toggle. “Customize” button opens visual editor. Base layer (blue) + equipment layer (teal).
6	**Review & Save**	Complete summary tree. Save inserts into normalized master tables. Triggers auto-generation of seed blocks and form templates.

Key Design Decisions

Disambiguation is mandatory. Even for “obvious” terms like HVAC. The system confirms what the user means before researching. Handles ambiguity (UPS = power backup or logistics?) and unknown terms.

Geography matters. Coastal locations get more frequent coil cleaning cycles. Humid climates get shorter drain flush cycles. Pharma industry context adds compliance checkpoints. Desert climates add sand/dust checkpoints.

Caching after first research. Research is one-time per equipment type per context. Cache key: equipment + climate zone + industry categories. Subsequent requests serve from Postgres.

“Add your own” at every level. Variants, spare parts, checkpoints, values, cycles — user can add custom items at each step. Custom items are tagged as user-contributed vs AI-researched.

2.2 Capability 3: Auto-Compose Form from Knowledge Tree

When equipment + nomenclature are selected during template/contract creation, the system auto-composes a smart form by querying the knowledge tree: SELECT checkpoints WHERE variant = X AND service_activity = Y, UNION with tenant custom checkpoints, order by section, render as form schema JSONB.

The form is not selected from a library — it is generated. This means every variant gets the right form automatically. A Split AC PM form is different from a Chiller PM form — different checkpoints, different readings, different thresholds.

The form renders in both web view (admin/office) and mobile view (technician in field). Same data, responsive layout.

2.3 Capability 4: Visual Form Editor

Tenants who need customization beyond the auto-generated form use a visual editor. This is NOT an NLP builder — it’s a drag-drop, toggle on/off, add/remove interface built on the existing form editor pattern at /admin/smart-forms/editor/:id.

The key difference from today: the editor starts from a pre-populated form (from the knowledge tree), not a blank canvas. The tenant's job is tweaking the last 20%, not building 100%.

2.4 Future Capabilities (Post-Launch)

Smart Recommender. Usage-pattern-based form recommendations. Requires tenant usage data. "72% of FM companies also attach Equipment Photo Log." Not buildable until 50+ tenants are active.

NLP Form Assistant. Natural language modifications: "add a section for site access verification." Layered on top of the visual editor as a power-user feature.

Network Effect Learning. When multiple tenants in pharma add "particulate count" as a checkpoint for HVAC, VaNi learns to include it by default for pharma context.

3. Skill Architecture

3.1 Knowledge Tree Skill — One Skill, Four Modes

The knowledge-tree-skill is a VaNi skill (SKILL.md + deterministic functions) that operates in different modes depending on the caller and context.

Mode	Who	What It Does	LLM	Output
Research	Admin	Generates full knowledge tree. Disambiguates, gathers context, invokes Claude API, presents for validation.	Claude API (one-time)	m_equipment_* tables
Registration	Tenant	Guides asset registration. Shows variant picker, auto-populates fields, suggests service schedules.	LFM2 (lightweight)	t_equipment entry
Form	System	Auto-composes smart form from knowledge tree. Filters by variant + activity + timing.	Not needed (deterministic)	m_form_templates
Alert	System	Monitors cycles, flags overdue checkpoints, detects reading trends, generates health scores.	LFM2 (summaries)	Alerts, reports

3.2 Skill Composition Hierarchy

The knowledge-tree-skill is the foundation. Other skills build on top of it:

knowledge-tree-skill → **catalog-studio-skill** (uses knowledge tree to build blocks, pricing, templates)

catalog-studio-skill → **template-skill + contract-skill + rfp-skill** (compose deliverables from catalog)

3.3 Intelligence Layer

Layer	When Used	Cost	Volume
Claude API	One-time research for new equipment types. ~175 calls to cover full catalog.	Minimal, amortized	~175 total

LFM2 (Self-Hosted)	Runtime: disambiguation, context overlay, registration guidance, trend summaries.	Zero API cost	Ongoing
Deterministic (No LLM)	Form composition, scheduling, threshold comparison, block generation. SQL + templates.	Zero	90% of ops

4. Data Model

4.1 New Master Tables (m_ prefix)

m_equipment_variants

Sub-types per equipment category. Linked to m_catalog_resource_templates (existing, 240 rows).

Column	Type	Notes
`id`	uuid PK	
`resource_template_id`	uuid FK	→ m_catalog_resource_templates.id
`name`	varchar(255)	e.g. "Split AC / Room AC"
`description`	text	Variant description
`capacity_range`	varchar(100)	e.g. "0.75–2.5 TR"
`attributes`	jsonb	Variant-specific metadata
`is_active, sort_order`	boolean, integer	Standard flags

m_equipment_spare_parts

Parts catalog grouped by component type (electrical, mechanical, refrigerant, filters, water-side, controls, consumables).

m_spare_part_variant_map

Junction table: which parts apply to which variants. The toggle matrix from the Knowledge Tree Builder.

m_equipment_checkpoints

What to check during service. Two types: condition (qualitative dropdowns with severity) and reading (quantitative with unit, normal range, thresholds). Includes service_activity (pm, repair, inspection, install, decommission), section_name (groups into form sections), and layer (base vs equipment-specific).

m_checkpoint_values

Valid answers for condition-type checkpoints. Each value has: label, severity (ok/attention/critical), triggers_part_consumption flag, requires_photo flag.

m_checkpoint_variant_map

Which checkpoints apply to which variants. Can include override_min/override_max for variant-specific thresholds.

m_service_cycles

Frequency per checkpoint, frequency_unit (days/hours/visits), varies_by factors (jsonb array), alert_overdue_days threshold.

m_context_overlays

Climate/geography/industry adjustments. Context type + value + adjustments jsonb. Allows the same knowledge tree to produce different cycle frequencies for different contexts.

4.2 Tenant Override Tables

t_custom_variants, t_custom_spare_parts, t_custom_checkpoints, t_custom_checkpoint_values, t_cycle_overrides. Each includes tenant_id. Merged with master data during form composition via UNION queries.

4.3 Enhanced Existing Tables

t_equipment (Registry)

New columns: variant_id (FK → m_equipment_variants), capacity_value (numeric), capacity_unit (varchar). Enables variant-aware registration guided by VaNi skill.

m_cat_blocks

New columns: form_template_id (FK → m_form_templates), knowledge_tree_ref (jsonb — {resource_template_id, variant_id}). The variant_pricing jsonb (currently empty) gets populated from knowledge tree variants.

m_form_templates

New columns: source (manual/knowledge_tree/tenant_fork), source_variant_id (FK → m_equipment_variants), service_activity (pm/repair/inspection/install/decommission).

t_form_templates (NEW — Tenant Forks)

Same schema as m_form_templates plus tenant_id, copied_from_id (FK → m_form_templates), is_forked (boolean). Created only when tenant customizes. Master updates don’t flow to forked copies. Future: “Master updated, want to merge?” notification.

4.4 Unchanged Tables

m_catalog_resource_templates (240 rows), m_catalog_industries (27), m_category_details (nomenclatures, block types), t_contracts, t_contract_blocks, m_form_template_mappings, m_form_submissions, t_cat_templates. No schema changes needed.

5. Integration Architecture

5.1 Knowledge Tree → Block Generation

When admin saves a Knowledge Tree, the system auto-generates seed blocks:

Service blocks (block_type = "service"): One per service activity (PM, Repair, Inspection, Install, Decommission). variant_pricing populated from knowledge tree. resource_pricing links to equipment. form_template_id links to auto-generated form.

Spare blocks (block_type = "spare"): One per spare part. config.sku auto-generated. config.inventory pre-populated. Variant mapping determines which spares show per variant.

Checklist blocks (block_type = "checklist" — currently 0 rows, defined but unused): Smart form wrapped as a block. THE BRIDGE between forms and blocks. form_template_id links to m_form_templates. Now forms are part of the block system.

All seed blocks: is_seed = true, tenant_id = NULL. Available to all tenants via existing RLS policies.

5.2 Registry Integration

The Knowledge Tree Skill in Registration Mode guides tenant equipment/facility registration:

1. Tenant types equipment name → disambiguation → variant picker from knowledge tree

2. Registration form auto-populates with variant-aware fields (make dropdown, capacity with correct unit, refrigerant type for HVAC)

3. INSERT into t_equipment with variant_id — asset is now linked to the full knowledge tree

4. VaNi suggests service schedules: “Based on your climate, filter check every 45 days”

Same pattern for facilities: Clean Room → ISO Class variant → facility-specific checkpoints.

5.3 Form Composition — Multiple Forms

A single contract can have multiple forms mapped via m_form_template_mappings:

Per equipment variant: Split AC PM Checklist ≠ Chiller PM Checklist (different checkpoints)

Per service activity: PM Checklist ≠ Breakdown Report ≠ Inspection Report

Per timing: Pre-service (safety check) + During (PM checklist) + Post-service (customer acknowledgment)

Per tenant: Master form OR tenant fork. 70–80% use master. Fork only when customizing.

5.4 Template → Contract Flow

Seed templates compose blocks (service + spare + checklist + text + billing) into reusable packages. Tenant copies template, adds pricing, publishes. Contract creation from template: blocks denormalized into t_contract_blocks (snapshot), forms mapped via m_form_template_mappings, equipment linked via t_contract_equipment.

5.5 Service Execution → Alerts

Technician opens service event → correct form loads (variant-specific, capacity-aware thresholds) → fills readings + condition checks → system validates against knowledge tree thresholds → submission saved to m_form_submissions → post-submission: trend analysis, cycle monitoring, overdue alerts, health scores.

6. Plan of Action — Phased Releases

The system is built incrementally. Each release is complete, reviewable, and functional before moving to the next. Claude Code executes under mentor guidance.

Release 1: Knowledge Tree Data Model + Seed Infrastructure

Objective: Build the foundation — all new tables, migrations, seed block types, and the data model that everything else depends on.

#	Task	Tables	Deliverable
1.1	Create m_equipment_variants table + migration	m_equipment_variants	Migration SQL
1.2	Create m_equipment_spare_parts + m_spare_part_variant_map	2 tables	Migration SQL
1.3	Create m_equipment_checkpoints + m_checkpoint_values + m_checkpoint_variant_map	3 tables	Migration SQL
1.4	Create m_service_cycles + m_context_overlays	2 tables	Migration SQL
1.5	Add new columns to t_equipment (variant_id, capacity)	ALTER TABLE	Migration SQL
1.6	Add new columns to m_cat_blocks (form_template_id, knowledge_tree_ref)	ALTER TABLE	Migration SQL
1.7	Add new columns to m_form_templates (source, variant_id, service_activity)	ALTER TABLE	Migration SQL
1.8	Create t_form_templates (tenant fork table)	t_form_templates	Migration SQL
1.9	Create tenant override tables (t_custom_*)	4 tables	Migration SQL
1.10	RLS policies for all new tables	All new tables	Migration SQL

Review checkpoint: All tables created, migrations run, RLS verified. No UI, no API yet.

Release 2: Knowledge Tree API + Claude Research Integration

Objective: Build the backend API that powers the Knowledge Tree Builder — research, disambiguation, CRUD for all knowledge tree entities.

#	Task	Endpoint	Deliverable
2.1	Claude API integration — structured prompt for equipment research	Internal service	ClaudeResearchService
2.2	Disambiguation endpoint — match input to existing catalog + suggest	POST /knowledge-tree/disambiguate	API + tests
2.3	Research endpoint — invoke Claude, return structured knowledge tree	POST /knowledge-tree/research	API + tests
2.4	CRUD for variants, spare parts, checkpoints, values, cycles	REST endpoints	API + tests
2.5	Context overlay engine — apply geo/climate/industry adjustments	Internal service	ContextOverlayService
2.6	Save knowledge tree — transactional insert across all tables	POST /knowledge-tree/save	API + tests

Review checkpoint: API functional, Claude research working, data saves correctly. Test with HVAC as first equipment type.

Release 3: Knowledge Tree Builder UI

Objective: Build the interactive Knowledge Tree Builder UX — the admin-facing tool. Based on the VaNi-branded prototype developed during design phase.

#	Task	Route	Deliverable

3.1	Step 0: Input + disambiguation + context animation	/admin/knowledge-tree	React component
3.2	Step 1: Variant selection with add-your-own		React component
3.3	Step 2: Spare parts matrix with per-variant toggles		React component
3.4	Step 3: Checkpoints editor (condition + reading types)		React component
3.5	Step 4: Service cycles with frequency/alert config		React component
3.6	Step 5: Form preview (web + mobile toggle)		React component
3.7	Step 6: Review summary + save		React component
3.8	Wire to Release 2 APIs		Integration

Review checkpoint: Admin can research HVAC end-to-end. Full knowledge tree saved. Seed first 5 equipment types: HVAC, Elevator, DG Set, Fire Alarm, UPS.

Release 4: Auto-Compose Forms + Seed Blocks

Objective: Knowledge tree data auto-generates smart forms and seed blocks. The bridge between knowledge tree and catalog studio.

#	Task	Component	Deliverable
4.1	Form composer service — SELECT checkpoints + variant filter → form JSONB	FormComposerService	Service + tests
4.2	Auto-generate m_form_templates entries per variant × service activity	FormGeneratorService	Service + tests
4.3	Seed block generator — create service + spare + checklist blocks	SeedBlockGenerator	Service + tests
4.4	Seed template generator — compose blocks into template	SeedTemplateGenerator	Service + tests
4.5	Trigger on knowledge tree save — auto-create forms + blocks + template	Post-save hook	Integration

Review checkpoint: Saving HVAC knowledge tree auto-creates PM forms for all variants, seed blocks, and a seed template. Verify in catalog studio.

Release 5: Registry Integration

Objective: Equipment and facility registration powered by the Knowledge Tree Skill in Registration Mode.

#	Task	Route/Component	Deliverable
5.1	Variant-aware registration form — variant picker + auto-populated fields	/settings/configure/resources	Enhanced UI
5.2	VaNi registration assistant — disambiguation + guidance during registration	VaNi sidebar	React component
5.3	Bulk registration — "I have 15 Split ACs across 3 floors"	Bulk add UI	React component

5.4	Service schedule suggestion post-registration	Post-registration flow	Integration
5.5	Facility registry — same pattern with facility variants	/settings/configure/resources	Enhanced UI

Review checkpoint: Tenant can register equipment with variant selection. Registered assets are linked to knowledge tree. Service schedules suggested.

Release 6: Form Customization + Tenant Forks

Objective: Tenant can customize auto-generated forms via visual editor. Fork mechanism for independent copies.

#	Task	Component	Deliverable
6.1	Fork mechanism — copy master form to t_form_templates on “Customize”	ForkService	Service + tests
6.2	Visual editor enhancement — pre-populated from knowledge tree	/admin/smart-forms/editor	Enhanced UI
6.3	Tenant custom checkpoints — add/remove fields in forked form	Editor UI	CRUD + UI
6.4	Tenant cycle overrides — adjust frequencies for their context	Settings UI	CRUD + UI

Review checkpoint: Tenant can use master form as-is OR fork and customize. Forked forms are independent. Visual editor works with pre-populated data.

Release 7: Contract Integration + Service Execution

Objective: Smart forms flow through contracts to service events. Technician fills variant-aware forms with real-time validation.

#	Task	Component	Deliverable
7.1	Template → contract form auto-mapping based on equipment + nomenclature	Contract wizard	Enhanced wizard
7.2	Multiple forms per contract — per equipment, per timing	Form mapping logic	Service + UI
7.3	Service event form loading — correct form for specific asset (variant-aware)	Service event page	Enhanced UI
7.4	Real-time validation — readings vs thresholds from knowledge tree	Form renderer	Enhanced FormRenderer
7.5	Mobile-responsive form fill experience	Form renderer	Responsive CSS

Review checkpoint: Full loop — template → contract → service event → form fill → submission with real-time validation.

Release 8: Alerts, Health Scores & LTV

Objective: Post-submission intelligence — the Alert Mode of the Knowledge Tree Skill.

#	Task	Component	Deliverable
8.1	Cycle monitoring — track overdue checkpoints per asset	Scheduled job	Cron + alerts

8.2	Reading trend analysis — compare last N submissions per asset	TrendAnalysisService	Service + UI
8.3	Equipment health scores — computed from submission data	HealthScoreService	Dashboard widget
8.4	LTV reports — “Your AMC covered 142 checkpoints, prevented 3 breakdowns”	LTVReportService	Report + PDF
8.5	Tenant notifications — overdue alerts, trend warnings	NotificationService	Email + in-app

Review checkpoint: Alerts firing, health scores visible, LTV report generated. The intelligence layer is live.

7. Release Summary & Timeline

Release	What	Depends On	Complexity	Priority
R1	Data Model + Migrations	—	Medium	P0 — Foundation
R2	Knowledge Tree API + Claude	R1	High	P0 — Foundation
R3	Knowledge Tree Builder UI	R2	High	P0 — Foundation
R4	Auto-Compose Forms + Seed Blocks	R3	Medium	P1 — Core
R5	Registry Integration	R1	Medium	P1 — Core
R6	Form Customization + Forks	R4	Medium	P2 — Enhancement
R7	Contract Integration + Execution	R4, R5	High	P2 — Enhancement
R8	Alerts, Health Scores, LTV	R7	Medium	P3 — Intelligence

R1–R3 are the foundation — must be built first. R4–R5 are the core product value. R6–R7 complete the contract loop. R8 adds the intelligence moat.

Each release ends with a review checkpoint. No release starts until the previous one is reviewed and approved. Claude Code builds incrementally under mentor guidance.

8. Appendix: Reference Materials

The following prototypes and documents were created during the design phase of this PRD:

1. Smart Forms Explorer — Interactive HTML showing industries, equipment, service activities, nomenclatures, and form mapping matrix.

2. HVAC PM Form (Web View) — Interactive HTML showing the layered form architecture (base + equipment layers) with 4 equipment sub-types (Split AC, VRF, Chiller, AHU).

3. Smart Forms Architecture (4 Capabilities) — Interactive HTML showing the original 4-capability architecture, intelligence stack, and data flows.

4. VaNi Knowledge Tree Builder — Interactive HTML prototype of the complete 7-step builder with HVAC data (variants, parts, checkpoints, cycles). VaNi branding.

5. Integration Architecture — Interactive HTML showing skill architecture (4 modes), data model (new + enhanced tables), block generation, registry integration, form composition, and end-to-end flow.

6. Smart Forms Complete Write-Up — Source document with equipment landscape (121 types), facility landscape (54 types), nomenclature definitions (21 types), and service activity mappings.

End of Document

VaNi Smart Forms — Vikuna Technologies — March 2026